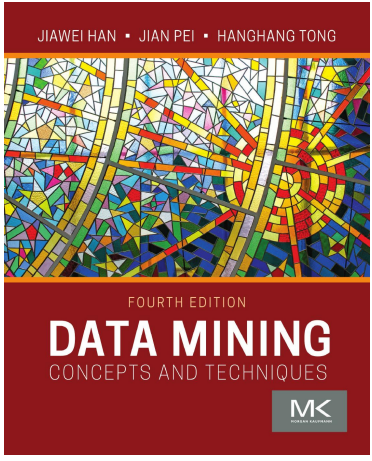
ML101
Chapter 4: Frequent Patterns
Data Mining: Concepts and Techniques

Jiawei Han, Jian Pei & Hanghang Tong

November 12, 2026

Data Mining: Concepts and Techniques



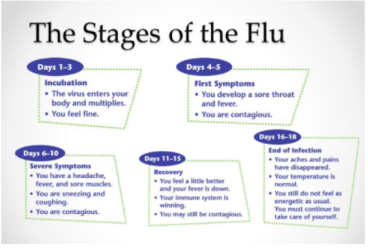
Content has been extracted from *Data Mining: Concepts and Techniques*, Fourth Edition, by Jiawei Han, Jian Pei & Hanghang Tong. Morgan Kaufmann Publishers. 2023.
Visit https://hanj.cs.illinois.edu/bk4/.

What Are Patterns?

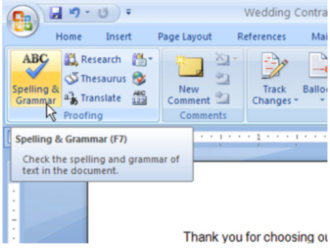
- **What are patterns?**
  - **Patterns:** A set of items, subsequences, or substructures that occur frequently together (or strongly correlated) in a data set
  - Patterns represent intrinsic and important properties of datasets



Frequent item set



Frequent sequences



Frequent structures

Plan

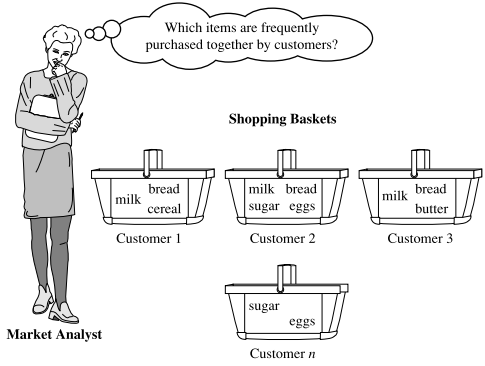
Basic Concepts

Frequent Itemset Mining Methods
- Apriori
- FPGrowth

Which Patterns Are Interesting? Pattern Evaluation Methods

What Is Pattern Discovery?

- **Pattern discovery:** Uncovering patterns from massive data sets
- It can answer questions such as:
  - What products were often purchased together?
  - What are the subsequent purchases after buying an iPad?



Pattern Discovery: Why Is It Important?

- **Foundation for many essential data mining tasks**
  - Association, correlation, and causality analysis
  - Mining sequential, structural (e.g., sub-graph) patterns
  - Classification: Discriminative pattern-based analysis
  - Cluster analysis: Pattern-based subspace clustering
- **Broad applications**
  - Market basket analysis, cross-marketing, catalog design, sale campaign analysis, Web log analysis, biological sequence analysis
  - Many types of data: spatiotemporal, multimedia, time-series, and stream data

Basic Concepts: Transactional Database

- Let $I = \{I_1, I_2, \ldots, I_m\}$ be a set of items
- **Transactional Database (TDB)**
  - Each transaction is associated with an identifier, called a **TID**.
  - May also have counts associated with each item sold

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Basic Concepts: k-Itemsets and Their Supports

- **Itemset:** A set of one or more items
- **k-itemset:** An itemset containing k items:

$$X = \{x_1, \ldots, x_k\}$$

  - Ex. {Beer, Nuts, Diaper} is a 3-itemset
- **Absolute support (count)**
  - sup$\{X\}$ = occurrences of an itemset X
  - Ex. sup{Beer} = 3, sup{Diaper} = 4
  - Ex. sup{Beer, Diaper} = 3
  - Ex. sup{Beer, Eggs} = 1

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- **Relative support (%)**
  - $s\{X\}$ = The fraction of transactions that contains X (i.e., the **probability** that a transaction contains X)
  - Ex. $s\{\text{Beer}\} = 3/5 = 60\%$
  - Ex. $s\{\text{Diaper}\} = 4/5 = 80\%$
  - Ex. $s\{\text{Beer, Eggs}\} = 1/5 = 20\%$

Basic Concepts: Frequent Itemsets (Patterns)

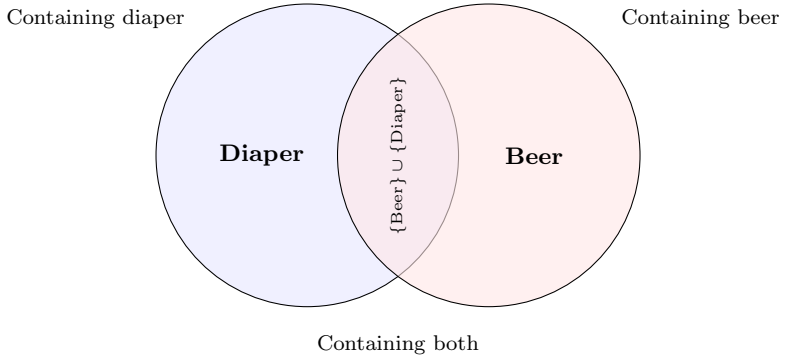
- An itemset (or a pattern) X is **frequent** if the support of X is no less than a minsup threshold σ
- Let $\sigma = 50\%$ (σ: minsup threshold) for the given 5-transaction dataset:
  - **All the frequent 1-itemsets:**
    - Beer: 3/5 (60%); Nuts: 3/5 (60%); Diaper: 4/5 (80%); Eggs: 3/5 (60%)
  - **All the frequent 2-itemsets:**
    - {Beer, Diaper}: 3/5 (60%)
  - **All the frequent 3-itemsets?**
    - None

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- Why do these itemsets (shown on the left) form the complete set of frequent k-itemsets (patterns) for any k?
- **Observation:** We may need an efficient method to mine a complete set of frequent patterns

From Frequent Itemsets to Association Rules

- Compared with itemsets, **association rules** can be more telling
  - Ex. Diaper $\rightarrow$ Beer: Buying diapers may likely lead to buying beers
- Note: $X \cup Y$: the union of two itemsets
  - The set contains both X and Y
  - $\{\text{Beer}\} \cup \{\text{Diaper}\} = \{\text{Beer, Diaper}\}$



Association Rules

- How do we compute the strength of an association rule $X \rightarrow Y$ (Both X and Y are itemsets)?
- We first compute the following two metrics, s and c:
  - **Support** of $X \cup Y$:
    - Ex. $s\{\text{Diaper, Beer}\} = 3/5 = 0.6$ (i.e., 60%)
  - **Confidence** of $X \rightarrow Y$:
    - The conditional probability that a transaction containing X also contains Y:

$$c = \frac{\sup(X, Y)}{\sup(X)}$$

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- Ex. $c = \frac{\sup\{\text{Diaper, Beer}\}}{\sup\{\text{Diaper}\}} = \frac{3}{4} = 0.75$

- In pattern analysis, we are often interested in those rules that dominate the database, and these two metrics ensure the **popularity** and **correlation** of X and Y.

Mining Frequent Itemsets and Association Rules

- **Association rule mining**
  - Given two thresholds: minsup, minconf
  - Find all of the rules, $X \rightarrow Y$ (s, c) such that $s \geq$ minsup and $c \geq$ minconf
- Let minsup $= 50\%$
  - Freq. 1-itemsets: Beer: 3, Nuts: 3, Diaper: 4, Eggs: 3
  - Freq. 2-itemsets: {Beer, Diaper}: 3
- Let minconf $= 50\%$
  - Beer $\rightarrow$ Diaper $(60\%, 100\%)$
  - Diaper $\rightarrow$ Beer $(60\%, 75\%)$
  - *(Q: Are these all the rules satisfying the two conditions?)*

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- **Observations:**
  - Mining association rules and mining frequent patterns are very close problems
  - Scalable methods are needed for mining large datasets

Challenge: There Are Too Many Frequent Patterns!

- A long pattern contains a combinatorial number of sub-patterns
- How many frequent itemsets does the following TDB_1 contain (minsup $= 1$)?
  - TDB_1: $T_1 : \{a_1, \ldots, a_{50}\}$; $\quad T_2 : \{a_1, \ldots, a_{100}\}$
- Let's have a try:
  - 1-itemsets: $\{a_1\} : 2, \{a_2\} : 2, \ldots, \{a_{50}\} : 2, \{a_{51}\} : 1, \ldots, \{a_{100}\} : 1$
  - 2-itemsets: $\{a_1, a_2\} : 2, \ldots, \{a_1, a_{50}\} : 2, \{a_1, a_{51}\} : 1, \ldots, \{a_{99}, a_{100}\} : 1$
  - $\ldots$
  - 99-itemsets: $\{a_1, a_2, \ldots, a_{99}\} : 1, \ldots, \{a_2, a_3, \ldots, a_{100}\} : 1$
  - 100-itemset: $\{a_1, a_2, \ldots, a_{100}\} : 1$
- The total number of frequent itemsets:

$$\binom{100}{1} + \binom{100}{2} + \cdots + \binom{100}{100} = 2^{100} - 1 \approx 1.27 \times 10^{30}$$

- **A too huge set for any one to compute or store!**

Expressing Patterns in Compressed Form: Closed Patterns

- **How to handle such a challenge?**
- **Solution 1: Closed patterns:** A pattern (itemset) X is **closed** if X is frequent, and there exists no super-pattern $Y \supset X$, with the same support as X
- Let Transaction DB TDB_1: $T_1 : \{a_1, \ldots, a_{50}\}$; $\quad T_2 : \{a_1, \ldots, a_{100}\}$
- Suppose minsup $= 1$. How many closed patterns does TDB_1 contain?
  - **Two:** P_1: "$\{a_1, \ldots, a_{50}\} : 2$"; $\quad P_2$: "$\{a_1, \ldots, a_{100}\} : 1$"
- **Closed pattern is a lossless compression of frequent patterns**
  - Reduces the # of patterns but does not lose the support information!
  - You will still be able to say: "$\{a_2, \ldots, a_{40}\} : 2$", "$\{a_5, a_{51}\} : 1$"

Expressing Patterns in Compressed Form: Max-Patterns

- **Solution 2: Max-patterns:** A pattern X is a **max-pattern** if X is frequent and there exists no frequent super-pattern $Y \supset X$
- **Difference from closed-patterns?**
  - Do not care the real support of the sub-patterns of a max-pattern
- Let Transaction DB TDB_1: $T_1 : \{a_1, \ldots, a_{50}\}$; $\quad T_2 : \{a_1, \ldots, a_{100}\}$
- Suppose minsup $= 1$. How many max-patterns does TDB_1 contain?
  - **One:** P: "$\{a_1, \ldots, a_{100}\} : 1$"
- **Max-pattern is a lossy compression!**
  - We only know $\{a_1, \ldots, a_{40}\}$ is frequent
  - But we do not know the real support of $\{a_1, \ldots, a_{40}\}, \ldots,$ any more!
- Thus in many applications, mining closed-patterns is more desirable than mining max-patterns

Plan

Basic Concepts

Frequent Itemset Mining Methods
- Apriori
- FPGrowth

Which Patterns Are Interesting? Pattern Evaluation Methods

Efficient Pattern Mining Methods

- The Downward Closure Property of Frequent Patterns
  - The Apriori Algorithm
  - Extensions or Improvements of Apriori
- Mining Frequent Patterns by Exploring Vertical Data Format
- FPGrowth: A Frequent Pattern-Growth Approach
- Mining Closed Patterns

The Downward Closure Property of Frequent Patterns

- **Observation:** From TDB_1: $T_1 : \{a_1, \ldots, a_{50}\}$; $T_2 : \{a_1, \ldots, a_{100}\}$
  - We get a frequent itemset: $\{a_1, \ldots, a_{50}\}$
  - Also, its subsets are all frequent: $\{a_1\}, \{a_2\}, \ldots, \{a_{50}\}, \{a_1, a_2\}, \ldots, \{a_1, \ldots, a_{49}\}, \ldots$
  - There are some hidden relationships among frequent patterns!
- The **downward closure (also called "Apriori")** property of frequent patterns:
  - If {**beer, diaper, nuts**} is frequent, so is {**beer, diaper**}
  - Every transaction containing {beer, diaper, nuts} also contains {beer, diaper}
  - **Apriori: Any subset of a frequent itemset must be frequent**.
- **Efficient mining methodology**
  - If any subset of an itemset S is infrequent, then there is no chance for S to be frequent, why do we even have to consider S!? **A sharp knife for pruning!**

Apriori Pruning and Scalable Mining Methods

- **Apriori pruning principle:** If there is any itemset which is infrequent, its superset should not even be generated! (Agrawal & Srikant @VLDB'94, Mannila, et al. @KDD'94)
- **Scalable mining Methods: Three major approaches**
  - **Level-wise, join-based approach:** Apriori (Agrawal & Srikant @VLDB'94)
  - **Vertical data format approach:** Eclat (Zaki, Parthasarathy, Ogihara, Li @KDD'97)
  - **Frequent pattern projection and growth:** FPGrowth (Han, Pei, Yin @SIGMOD'00)

Apriori: A Candidate Generation & Test Approach

- **Outline of Apriori (level-wise, candidate generation and test)**
  - Scan DB once to get frequent 1-itemset
  - **Repeat**
    - Generate length-$(k+1)$ candidate itemsets from length-k frequent itemsets
    - Test the candidates against DB to find frequent $(k+1)$-itemsets
    - Set $k := k+1$
  - **Until** no frequent or candidate set can be generated
  - Return all the frequent itemsets derived

The Apriori Algorithm (Pseudo-Code)

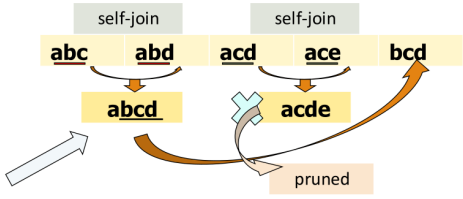
- C_k: Candidate itemset of size k
- F_k: Frequent itemset of size k

Apriori Algorithm

```
k := 1;
F_k := {frequent items};                                  // frequent 1-itemset
While (F_k ≠ ∅) do {                                      // when F_k is non-empty
    C_{k+1} := candidates generated from F_k;             // candidate generation
    Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;
    k := k + 1;
}
return ⋃_k F_k;                                           // return F_k generated at each level
```

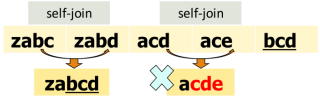
Apriori: Implementation Tricks

- **How to generate candidates?**
  - Step 1: self-joining F_k
  - Step 2: pruning
- **Example of candidate-generation**
  - $F_3 = \{abc, abd, acd, ace, bcd\}$
  - **Self-joining:** $F_3 * F_3$
    - $abcd$ from abc and abd
    - $acde$ from acd and ace
  - **Pruning:**
    - $acde$ is removed because ade is not in F_3
  - $C_4 = \{abcd\}$



prefix matching

Candidate Generation (Pseudo-Code)



- Suppose the items in F_{k-1} are listed in an order

Candidate generation

```
// Step 1:  Joining
for each p in F_{k-1}
  for each q in F_{k-1}
    if p.item_1 = q.item_1, ..., p.item_{k-2} = q.item_{k-2},
      p.item_{k-1} < q.item_{k-1} {
       c = join(p, q)
       // Step 2:  pruning
       if has_infrequent_subset(c, F_{k-1})
          continue                                          // prune
       else add c to C_k
    }
```

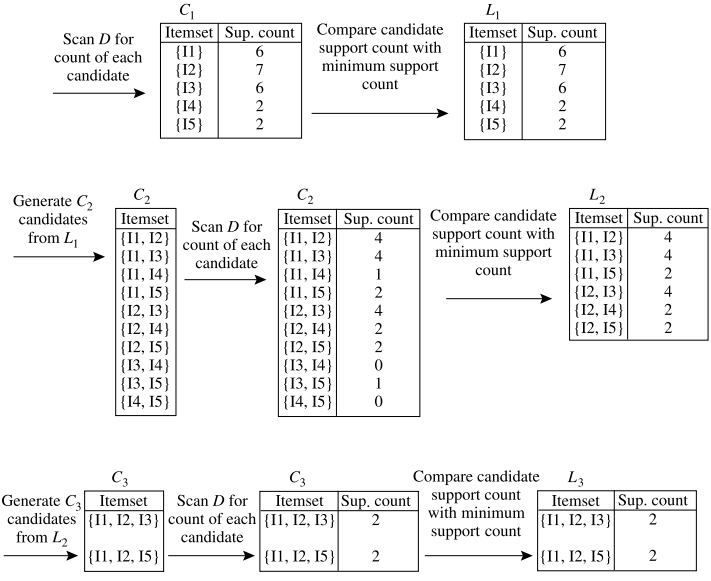
The Apriori Algorithm –An Example

Table 4.1 A transactional data set.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Database TDB (minsup = 2)

The Apriori Algorithm –An Example



Apriori: Improvements and Alternatives

- **Reduce passes of transaction database scans**
  - Partitioning (e.g., Savasere, et al., 1995)
  - Dynamic itemset counting (Brin, et al., 1997)
- **Shrink the number of candidates**
  - Hashing (e.g., DHP: Park, et al., 1995)
  - Pruning by support lower bounding (e.g., Bayardo, 1998)
  - Sampling (e.g., Toivonen, 1996)
- **Exploring special data structures**
  - Tree projection (Agarwal, et al., 2001)
  - H-miner (Pei, et al., 2001)
  - Hypercube decomposition (e.g., LCM: Uno, et al., 2004)

Why Mining Frequent Patterns by Pattern Growth?

Apriori: A breadth-first search mining algorithm

- First find the complete set of frequent k-itemsets
- Then derive frequent $(k+1)$-itemset candidates
- Scan DB again to find true frequent $(k+1)$-itemsets

Motivation for a different mining methodology

- Can we develop a **depth-first search** mining algorithm?
- For a frequent itemset ρ, can subsequent search be confined to only those transactions containing ρ?

Such thinking leads to a frequent pattern growth approach: **FPGrowth**

FPGrowth (J. Han, J. Pei, Y. Yin, "Mining Frequent Patterns without Candidate Generation," *SIGMOD 2000*)

Example: From Transactional DB to Ordered Frequent Itemlist

Let min_support $= 3$

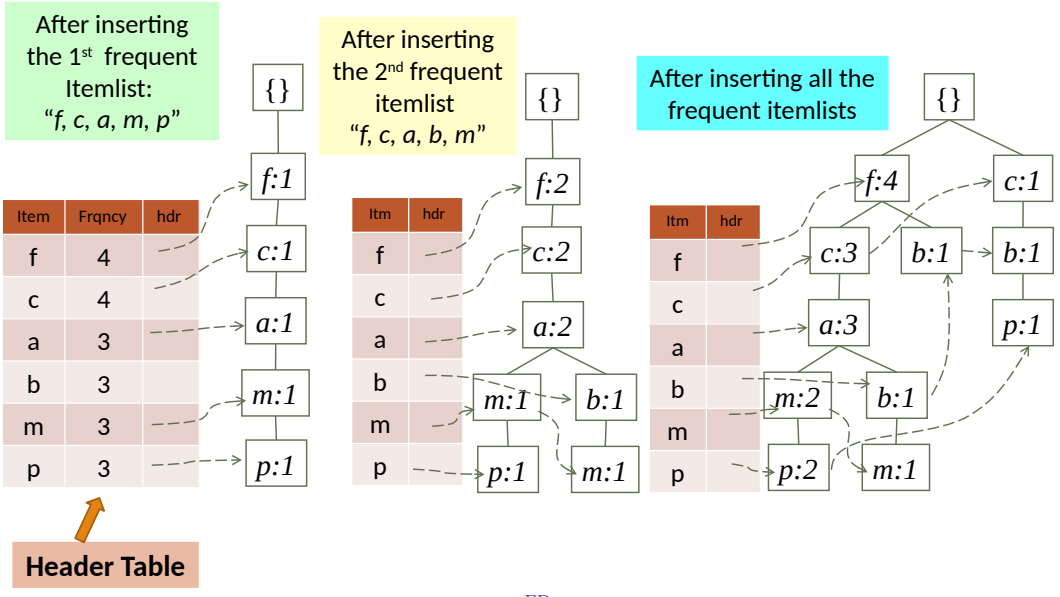
- **Scan DB once**, find single-item frequent patterns: `f:4, a:3, c:4, b:3, m:3, p:3`
- **Sort frequent items** in frequency-descending order (f-list): **F-list** = `f-c-a-b-m-p`
- **Scan DB again**, use the ordered frequent itemlist for each transaction to construct an FP-tree.

TID	Items in Transaction	Ordered Freq. Itemlist
100	$\{f, a, c, d, g, i, m, p\}$	f, c, a, m, p
200	$\{a, b, c, f, l, m, o\}$	f, c, a, b, m
300	$\{b, f, h, j, o, w\}$	f, b
400	$\{b, c, k, s, p\}$	c, b, p
500	$\{a, f, c, e, l, p, m, n\}$	f, c, a, m, p

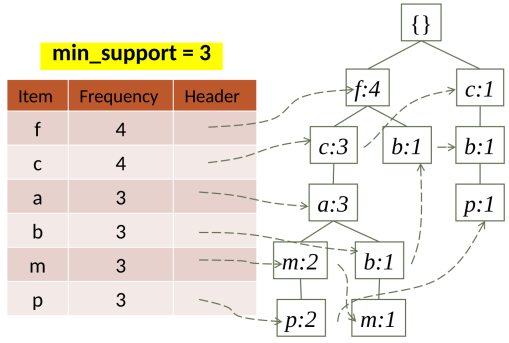
FP-Tree Construction:

For each transaction,insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated.

Example: Construct FP-tree from Transaction DB



Mining FP-Tree: Divide and Conquer Based on Patterns and Data



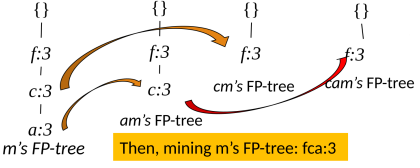
Partitioning Pattern Mining

- Pattern mining can be partitioned according to current patterns:
  - **Patterns containing p:** p's conditional database: `fcam:2, cb:1`
    - p's conditional database (i.e., the database under the condition that p exists):
    - *transformed prefix paths of item p*
  - **Patterns having m but no p:** m's conditional database: `fca:2, fcab:1`

Mine Each Conditional Database Recursively

min_support = 3

Item	Conditional Database
c	`f:3`
a	`fc:3`
b	`fca:1, f:1, c:1`
m	`fca:2, fcab:1`
p	`fcam:2, cb:1`



For Each Conditional Database

- Mine single-item patterns
- Construct its conditional FP-tree & mine it

Conditional FP-Tree Results

- p**'s conditional DB:** `fcam:2, cb:1` $\rightarrow$ `c:3`
- m**'s conditional DB:** `fca:2, fcab:1` $\rightarrow$ `fca:3`
- b**'s conditional DB:** `fca:1, f:1, c:1` $\rightarrow \emptyset$

Actually, for single branch FP-tree, all the frequent patterns can be generated in one shot

FPGrowth: Mining Frequent Patterns by Pattern Growth

Essence of Frequent Pattern Growth (FPGrowth) Methodology

- Find frequent single items and partition the database based on each such single-item pattern
- Recursively grow frequent patterns by doing the above for each partitioned database (also called the pattern's **conditional database**)
- To facilitate efficient processing, an efficient data structure, **FP-tree**, can be constructed

Mining Process

- **Recursively construct and mine** (conditional) FP-trees
- **Termination condition:** Until the resulting FP-tree is empty, or until it contains only **one path**, a single path will generate all the combinations of its sub-paths, each of which is a frequent pattern

Plan

Basic Concepts

Frequent Itemset Mining Methods
- Apriori
- FPGrowth

Which Patterns Are Interesting? Pattern Evaluation Methods

How to Judge if a Rule/Pattern Is Interesting?

The Challenge

- Pattern-mining will generate a large set of patterns/rules
- **Not all** the generated patterns/rules are interesting

Interestingness Measures: Objective vs. Subjective

Objective Interestingness Measures

- Data-driven statistical metrics:
  - Support
  - Confidence
  - Correlation, ...

Subjective Interestingness Measures

- Different users may judge interestingness differently
- Let a user specify:
  - **Query-based:** Relevant to a user's particular request
  - **Against knowledge-base:** Unexpected, freshness, timeliness

Limitation of the Support-Confidence Framework

Are s and c interesting in association rules: "$A \Rightarrow B$" $[s, c]$?

Be careful! High support and confidence can still produce misleading rules.

Example: School Student Statistics

	play-bball	**not play-bball**	**sum (row)**
eat-cereal	400	350	750
not eat-cereal	200	50	250
sum (col.)	600	400	1000

Association Rule Analysis

- **Generated Rule:** play-basketball $\Rightarrow$ eat-cereal $[40\%, 66.7\%]$ *(higher s & c)*
- **Why it is misleading:** Overall % of students eating cereal is **75**% $>$ **66.7**% (playing basketball actually lowers the likelihood!).
- **A more telling rule:** $\neg$play-basketball $\Rightarrow$ eat-cereal $[35\%, 87.5\%]$ *(high s & c)*

Interestingness Measure: Lift

Measure of Dependent/Correlated Events

$$\text{lift}(B, C) = \frac{c(B \to C)}{s(C)} = \frac{s(B \cup C)}{s(B) \times s(C)}$$

Lift(B, C) may tell how B and C are correlated

- $\text{lift}(B, C) = 1$: B and C are **independent**
- $\text{lift}(B, C) > 1$: **positively** correlated
- $\text{lift}(B, C) < 1$: **negatively** correlated

For Our Example (B: basketball, C: cereal)

$$\text{lift}(B, C) = \frac{400/1000}{(600/1000) \times (750/1000)} = \mathbf{0.89}$$

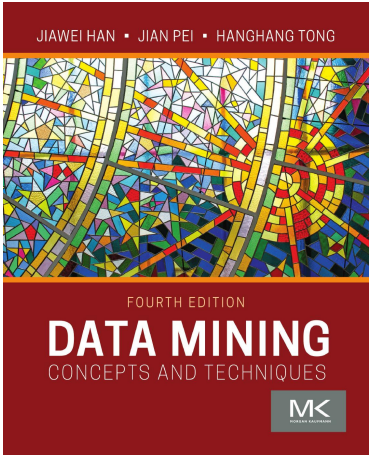
$$\text{lift}(B, \neg C) = \frac{200/1000}{(600/1000) \times (250/1000)} = \mathbf{1.33}$$

Conclusion

- B and C are **negatively correlated** since $\text{lift}(B, C) < 1$
- B and $\neg C$ are **positively correlated** since $\text{lift}(B, \neg C) > 1$

End of Chapter 4.

Data Mining: Concepts and Techniques



Content has been extracted from *Data Mining: Concepts and Techniques*, Fourth Edition, by Jiawei Han, Jian Pei & Hanghang Tong. Morgan Kaufmann Publishers. 2023.
Visit https://hanj.cs.illinois.edu/bk4/.